

DBMS Indexing Numerical

Search Complexity & Disk I/O Cost Without Indexing

1. GIVEN PROBLEM STATEMENT & PARAMETERS

In Database Management Systems (DBMS), data resides on secondary storage (hard disk) divided into physical blocks. Searching for a specific record requires fetching these blocks into RAM.

BLOCK SIZE (B) 1,000 Bytes	RECORD SIZE (R) 250 Bytes	TOTAL RECORDS (N) 10,000	DATA STATE Unordered
-------------------------------	------------------------------	-----------------------------	-------------------------

2. STEP-BY-STEP BLOCK CALCULATION

Step A: Records Per Block

The number of records that can fit into a single disk block is determined by dividing the block size by the record size:

Blocking Factor (Bfr) = Block Size / Record Size = 1000 / 250 = **4 records / block**

Step B: Total Number of Blocks (N)

To store all 10,000 records, the required total number of disk blocks is calculated as:

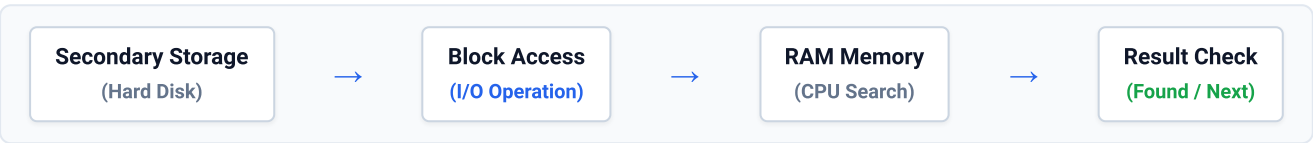
Total Blocks (N) = Total Records / Records per Block = 10,000 / 4 = **2,500 blocks**

3. THE NEED FOR BLOCK-BY-BLOCK SEARCH

Consider executing a simple SQL query on the Student table:

```
SELECT * FROM Student WHERE Roll_No = 345;
```

Because the entire database is usually too large to fit into RAM all at once, the DBMS transfers blocks from the disk to main memory sequentially to perform inspection.



4. COST ANALYSIS FOR UNORDERED DATA

Since the records are **unordered**, there is no organizational structure to inform us where record Roll_No = 345 resides. We must perform a linear search across the blocks.

Case	Scenario Description	Block Accesses (I/O Cost)
Best Case	The target record is located inside the very first block accessed.	1 block access
Worst Case	The target record is in the final block (or does not exist at all).	2,500 block accesses
Average Case	Calculated as N / 2 for linear search over N total blocks.	1,250 block accesses

5. ORDERED DATA VS. BINARY SEARCH

If the table records are pre-sorted (ordered) on disk by the search key (e.g., Roll_No), the DBMS can utilize **Binary Search** instead of linear scanning.

Binary Search Complexity: $O(\log_2 N)$
For $N = 2,500$ blocks: $\log_2(2500) \approx 11.28 \rightarrow \approx 12$ block accesses

6. COMPARATIVE ANALYSIS

Comparing performance across data arrangements illustrates the reduction in secondary storage I/O operations:

Data Arrangement	Search Method	Average Search Cost	Time Complexity
Unordered Data	Linear Search	1,250 block accesses	$O(N)$
Ordered Data	Binary Search	≈ 12 block accesses	$O(\log N)$
With Indexing	Index Lookup	$\approx 2 - 4$ block accesses	$O(\log_B N)$

Key Takeaway: Secondary Storage I/O vs CPU Overhead

In database systems, disk I/O (fetching blocks) is orders of magnitude slower than CPU operations (comparing records inside RAM). Thus, search efficiency is measured primarily in **block accesses** rather than individual record comparisons.

7. COMPLETE NUMERICAL SOLUTION SUMMARY

- Given:** Block Size = 1000 B, Record Size = 250 B, Records = 10,000 (Unordered)
- Records / Block:** $1000 / 250 = 4$
 - Total Blocks (N):** $10,000 / 4 = 2,500$
 - Best Case:** 1 block access
 - Worst Case:** 2,500 block accesses
 - Average I/O Cost:** $2500 / 2 = 1,250$ block accesses
 - Search Complexity:** $O(N)$

8. EXAM FORMULA SHEET & MEMORY TRICK

Standard Formulas

- Records per block (Bfr)** = $\lfloor \text{Block Size (B)} / \text{Record Size (R)} \rfloor$
- Total Blocks (N)** = $\lceil \text{Total Records (n)} / \text{Bfr} \rceil$
- Unordered Search Cost** = $N / 2$ (Average), N (Worst)
- Ordered Search Cost** = $\lceil \log_2(N) \rceil$

Memory Rule: "Divide sizes $\rightarrow$ Divide records $\rightarrow$ Search blocks."
 $1000 \div 250 = 4$ records/block $\Rightarrow 10,000 \div 4 = 2,500$ blocks $\Rightarrow 2,500 \div 2 = 1,250$ accesses

